

Cloud Operations Lab

[image] [image] [image] [image]

An AWS infrastructure project built to demonstrate Cloud Engineering skills end-to-end: Infrastructure as Code, automated monitoring, operational automation, and a full CI/CD deployment pipeline with manual approval gates.

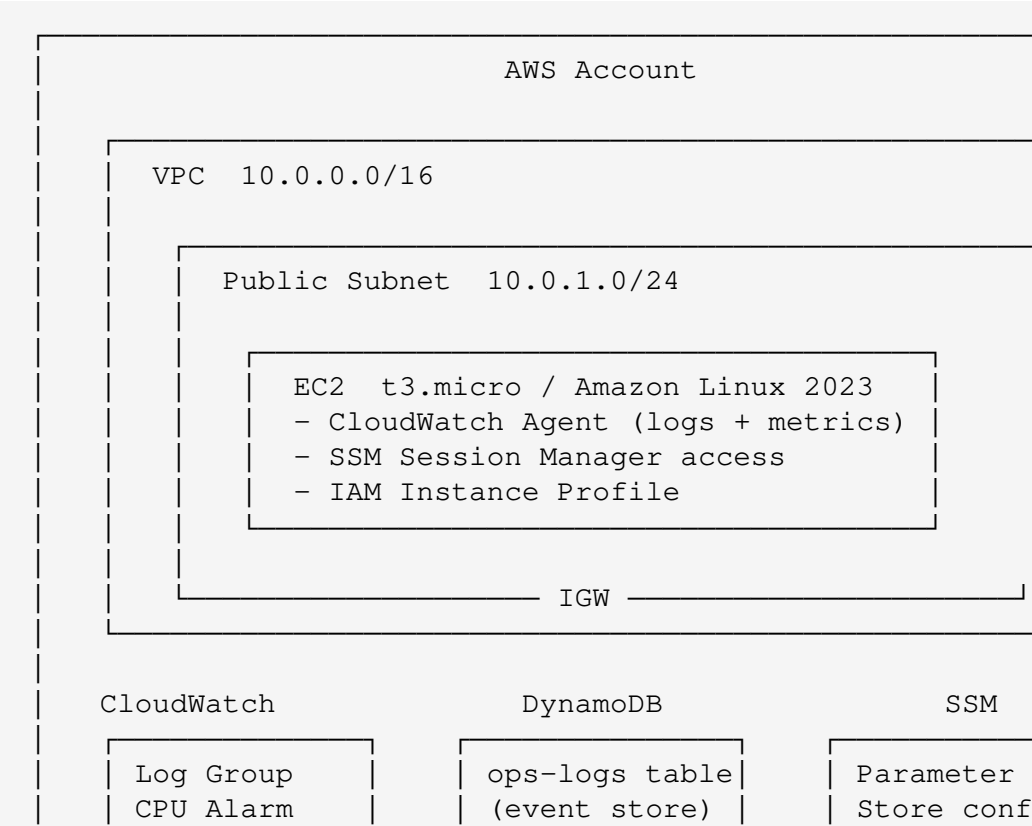
The project was built incrementally in five sprints, each adding a capability layer on top of the last.

What This Project Demonstrates

Skill Area	Implementation
Infrastructure as Code	Modular Terraform with remote state (S3 + DynamoDB locking)
Cloud Networking	VPC, public subnet, Internet Gateway, route table, security groups
Compute	EC2 on Amazon Linux 2023 with IMDSv2 instance metadata
Secure Remote Access	SSM Session Manager — no SSH, no open ports, no key pairs
Identity & Access Management	Least-privilege IAM roles, scoped inline policies, instance profiles
Observability	CloudWatch Agent, structured log collection, CPU alarm, SNS email alerts

Operational Automation	SSM Run Command documents, bash scripts, DynamoDB operational event log
CI/CD	GitHub Actions with OIDC — no long-lived AWS credentials anywhere
Deployment Control	GitHub Environments, manual approval gate before terraform apply
Security	OIDC trust scoped to specific environments, iam:PassRole scoped to EC2 service only

Architecture



SNS → Email

Run Command
Documents

S3 (Terraform state)
GitHub OIDC Provider

DynamoDB (Terraform logs)
CI + Apply IAM Roles

CI/CD Pipeline

The full deployment flow:

Developer pushes code



Pull Request



terraform fmt

← fails if code is not correct



terraform init



terraform validate



terraform plan

← full output posted as a PR comment



Code Review + Plan Review



Merge to main



Apply workflow triggered



← GitHub Environment "dev" – job pauses here

Manual Approval required

(designated reviewer clicks Approve)



```
terraform apply
```

Why OIDC instead of AWS Access Keys

GitHub OIDC lets GitHub Actions request a short-lived, automatically expiring AWS credential tied to a specific job. There are no long-lived secrets stored anywhere — nothing to rotate, nothing to accidentally leak. If the OIDC token is somehow obtained, it is already expired by the time it could be misused.

Two separate IAM roles, different trust conditions

Role	When it can be assumed	Permissions
github-ci	Any workflow in this repository (PRs, any branch)	ReadOnlyAccess + state backend
github-apply	Only workflows running inside the dev GitHub Environment	Custom write policy + state backend

When a job declares `environment: dev` in GitHub Actions, the OIDC token's `sub` claim changes from `repo:{org}/{repo}:ref:refs/heads/...` to `repo:{org}/{repo}:environment:dev`. The `apply` role trust policy matches only this second form using `StringEquals`. A rogue workflow that omits the `environment` declaration produces a different `sub` and cannot assume the `apply` role — even if it knows the ARN. This is enforced cryptographically by AWS, not just by workflow logic.

Repository Structure

```
cloud-operations-lab/
```

```
├─ bootstrap/                                # One-time setup – runs
│   ├── main.tf                             # S3 state bucket, Dyna
│   │                                         # GitHub OIDC provider,
│   ├── variables.tf
│   ├── outputs.tf
│   ├── terraform.tfvars                   # Your values (gitignor
│   └─ terraform.tfvars.example            # Template with instruc
├─ modules/                                # Reusable infrastru
│   ├── vpc/                               # VPC, public subnet, IGW, route tabl
│   ├── iam/                               # EC2 role, instance profile, scoped
│   ├── ec2/                               # Instance, AMI data source, CloudWat
│   ├── cloudwatch/                       # Log group, CPU utilisation alarm, S
│   ├── dynamodb/                         # Ops-logs table (PK: instance_id, SK
│   └─ ssm/                               # Parameter Store config entry, Run C
├─ environments/
│   └─ dev/                                # Dev environment – wir
│       ├── backend.tf                     # S3 remote state (key:
│       ├── providers.tf                  # AWS provider with pro
│       ├── main.tf                       # Module composition
│       ├── variables.tf
│       ├── outputs.tf
│       ├── dev.tfvars                    # Environment values (c
│       └─ dev.tfvars.example             # Template for forks /
├─ scripts/
│   └─ bash/
│       ├── health_check.sh               # Collects instance met
│       └─ log_event.sh                   # Writes a named operat
└─ .github/
    └─ workflows/
        ├── terraform-plan.yml            # Triggered on Pull Req
        └─ terraform-apply.yml            # Triggered on merge to
```

Sprint Build Log

The project was built in five sprints. Each sprint is independently testable.

Sprint 1 — Infrastructure Foundation

Establishes the networking, compute, and secure access skeleton.

- **Terraform remote state:** S3 bucket (versioned, all public access blocked) stores the state file. DynamoDB provides lock acquisition so concurrent runs cannot corrupt state.
- **VPC:** Isolated network in `us-east-1` with a public subnet, Internet Gateway, and an explicit route table.
- **Security Group:** Egress-only — no inbound ports open at all. SSM Session Manager connects outbound over HTTPS to the SSM service endpoint. Port 22 is never needed.
- **EC2:** `t3.micro` running Amazon Linux 2023. No SSH key pair is generated or stored. The only way to access the instance is through SSM Session Manager.
- **IAM:** Instance role with the `AmazonSSMManagedInstanceCore` managed policy. This is the minimum permission set for SSM to function.

Sprint 2 — CloudWatch Monitoring

Adds observability so you can see what the instance is doing without accessing it.

- **CloudWatch Agent:** Installed and configured via EC2 `user_data`. Collects system logs and instance metrics.
- **Logging fix for Amazon Linux 2023:** AL2023 uses `systemd-journald` and does not write to `/var/log/messages` or `/var/log/secure` by default. `rsyslog` is installed in `user_data` to bridge `journald` output into those files, which the CloudWatch Agent then reads.
- **CPU alarm:** Triggers when utilisation stays above 80% for two

consecutive 5-minute periods. Designed to catch runaway processes, not momentary spikes.

- **SNS topic:** Receives the alarm signal and sends an email notification. Requires inbox confirmation before alerts are delivered.
- **IAM update:** `CloudWatchAgentServerPolicy` added to the instance role.

Sprint 3 — Operational Automation

The instance can now report its own health and log operational events to a persistent, queryable store.

- **DynamoDB ops-logs table:** Each item is a structured event keyed by `instance_id` (partition) and `log_timestamp` (sort key, ISO 8601). A TTL attribute automatically expires entries after 30 days — the table never needs manual cleanup.
- **SSM Parameter Store:** Stores the DynamoDB table name and region. Scripts look up their target at runtime from this parameter, so they work correctly across environments without any hard-coded values.
- **SSM Run Command documents:** Two documents embed the bash scripts and can be executed from the AWS Console or CLI without any shell access to the instance.
 - `health-check`: Collects CPU idle percentage, load average, available memory, and disk usage, then writes a structured item to DynamoDB.
 - `log-event`: Writes a named event (e.g. `deployment`, `maintenance`) with an optional message to DynamoDB.
- **Windows line-ending fix:** The bash scripts are developed on Windows (CRLF line endings). The shebang line `#!/bin/bash\r` is not recognised by the Linux kernel, causing the script to fail with "required file not found". Terraform normalises line endings to LF using `replace()` before embedding the script content in the SSM document.
- **IAM count fix:** The ops-automation inline policy uses `count = var.enable_ops_automation ? 1 : 0`. Using the DynamoDB table ARN (a computed value) directly in `count` would fail at plan time because Terraform cannot determine `count` until

apply. A boolean variable is known at plan time and avoids this error entirely.

- **IAM update:** Scoped inline policy adds `dynamodb:PutItem` (specific table ARN only) and `ssm:GetParameter` (specific parameter path only).

Sprint 4 — GitHub Actions CI

Every Pull Request gets an automated Terraform plan. No engineer needs to run Terraform locally to review an infrastructure change.

- **GitHub OIDC Provider:** Registered once in the AWS account during bootstrap. GitHub Actions presents a signed JWT; AWS verifies it against this provider and issues temporary credentials.
- **CI IAM Role:** `ReadOnlyAccess` is sufficient for `terraform plan` since it never writes infrastructure. Trust policy is scoped to this specific repository's OIDC subject.
- **Plan output capture fix:** `steps.*.outputs.stdout` is unreliable for large outputs or when `continue-on-error: true` is set. The plan step redirects output to `/tmp/tfplan.txt` using `tee` and `PIPESTATUS` (to preserve the exit code through the pipe). The PR comment step reads directly from that file.
- **.gitignore precision fix:** The initial `*.tfvars` rule ignored `dev.tfvars`, preventing CI from reading environment variables. Fixed to ignore only auto-loaded files (`terraform.tfvars`, `*.auto.tfvars`) while allowing named environment files like `dev.tfvars` to be committed and tracked.

Sprint 5 — Controlled Deployment

`terraform apply` now requires a merge to `main` AND explicit human approval. No infrastructure is ever changed automatically.

- **GitHub Environment:** A protected environment named `dev` is configured in repository settings with required reviewers. The apply job cannot start without approval.
- **Apply IAM Role:** A separate role with write permissions across all managed services (EC2/VPC, IAM, DynamoDB, CloudWatch,

SNS, SSM). Its trust policy requires the OIDC `sub` claim to be `repo:{org}/{repo}:environment:dev` — a value that is only present when a job explicitly runs inside the protected environment.

- **iam:PassRole scoping:** The most sensitive IAM permission is double-restricted: it can only pass roles whose names match the project naming convention (`cloud-ops-lab-*`), and only to the EC2 service. It cannot be used to escalate privileges to any other AWS service.
 - **-auto-approve is safe here:** The plan was reviewed during the PR. Manual approval was given through the GitHub Environment gate. The apply job uses `-auto-approve` because the human decision point is the approval step, not a Terraform prompt.
-

Getting Started

Prerequisites

- [AWS CLI v2](#) configured with credentials that have broad permissions (only needed for the one-time bootstrap)
- [Terraform](#) `>= 1.5`
- A GitHub repository (this project or your fork)
- An AWS account — all resources are within Free Tier limits

Step 1 — Bootstrap (run once)

The bootstrap uses **local** Terraform state and creates the shared infrastructure that all environments depend on: the S3 state bucket, the DynamoDB lock table, the GitHub OIDC provider, and the two CI/CD IAM roles.

```
cd bootstrap
cp terraform.tfvars.example terraform.tfvars
# Fill in your values in terraform.tfvars
```

```
terraform init
terraform plan
terraform apply
```

Save the outputs — you will need the two role ARNs in the next step:

```
terraform output ci_role_arn
terraform output apply_role_arn
```

Step 2 — Add GitHub Secrets

Go to your repository → **Settings** → **Secrets and variables** → **Actions**

Secret name	Value
AWS_CI_ROLE_ARN	Output of <code>terraform output ci_role_arn</code>
AWS_APPLY_ROLE_ARN	Output of <code>terraform output apply_role_arn</code>

Step 3 — Create the GitHub Environment

Go to your repository → **Settings** → **Environments** → **New environment**

- Name: `dev` (exact, lowercase — the workflow and IAM trust policy both match this string)
- Enable **Required reviewers** and add yourself
- Click **Save protection rules**

Step 4 — Deploy via Pull Request

```
git checkout -b feature/initial-deploy
# Make any change to trigger the workflow
git add .
```

```
git commit -m "chore: trigger initial deploy"
git push -u origin feature/initial-deploy
```

Open a Pull Request targeting `main`. The plan workflow runs automatically and posts the full Terraform plan as a PR comment. Review it, then merge.

After merge, the apply workflow triggers. It pauses at the environment gate. Go to **Actions**, click the pending run, then **Review deployments** → **Approve and deploy**.

Step 5 — Access the instance (no SSH required)

```
cd environments/dev
terraform output instance_id

aws ssm start-session --target <instance-id>
```

Step 6 — Run the automation scripts

From the CLI:

```
# Health check - collects metrics and writes a DynamoDB i
aws ssm send-command \
  --document-name "cloud-ops-lab-dev-health-check" \
  --targets "Key=instanceids,Values=<instance-id>"

# Log a custom event
aws ssm send-command \
  --document-name "cloud-ops-lab-dev-log-event" \
  --parameters '{"EventType":["deployment"],"Message":["R
  --targets "Key=instanceids,Values=<instance-id>"
```

View results: **AWS Console** → **DynamoDB** → **Tables** → **cloud-ops-lab-dev-ops-logs** → **Explore items**

Tear down

```
cd environments/dev
terraform destroy -var-file="dev.tfvars"

# Remove bootstrap resources (optional)
cd ../../bootstrap
terraform destroy
```

Key Design Decisions

No SSH, no key pairs. SSM Session Manager provides interactive shell access over HTTPS. No inbound port is ever open. There is no private key to manage, rotate, or accidentally commit to Git.

Remote state with locking. State lives in S3 with versioning enabled. DynamoDB provides atomic lock acquisition. If a `terraform apply` is interrupted, the lock prevents a second run from starting until the first lock is explicitly released or expires.

Modular Terraform. Each module has exactly one responsibility. `environments/dev/main.tf` is a composition layer — it wires modules together with environment-specific values but contains no resource definitions itself. Adding a `prod` environment means writing a new composition with different inputs, reusing the same modules.

count driven by a boolean, not a computed ARN. The ops-automation IAM policy is conditional. Using `count = var.dynamodb_table_arn != null ? 1 : 0` fails at plan time because the ARN is a computed value (only known after apply). Using `count = var.enable_ops_automation ? 1 : 0` works because a boolean input variable is always known at plan time.

OIDC sub claim scoped to a GitHub Environment. The apply IAM role trust policy uses `StringEquals` (not `StringLike`) to match `repo:{org}/{repo}:environment:dev`. This value is only present

in the OIDC token when the job explicitly declares `environment: dev` — which triggers the GitHub Environment protection rules. The two gates (approval + cryptographic OIDC condition) are independent and both must pass.

AWS Free Tier Compatibility

Resource	Free Tier limit	This project uses
EC2	750 hrs/month (t2.micro or t3.micro)	1 instance
S3	5 GB storage, 20,000 GET requests	< 1 MB total
DynamoDB	25 GB storage, 200M requests/month	2 tables, minimal traffic
CloudWatch	10 custom metrics, 5 GB log ingestion/month	1 alarm, system logs
SSM	Free for Standard tier parameters and Run Command	Standard tier only
SNS	1M publishes/month	1 topic, alarm-triggered

Note: `t3.micro` is Free Tier eligible on accounts created after December 2024. If your account qualifies only for `t2.micro`, update `instance_type` in `environments/dev/dev.tfvars`.

Troubleshooting

terraform init fails — module does not exist All six module directories (`vpc`, `iam`, `ec2`, `cloudwatch`, `dynamodb`, `ssm`) must be

present. If any are missing, the repository was cloned from an incomplete commit. Pull the latest commits from all branches.

CloudWatch logs not appearing in the console Amazon Linux 2023 does not write to `/var/log/messages` without `rsyslog`. The `user_data` script installs it, but `user_data` only runs on first boot. If the instance already existed before this change, it must be replaced: `terraform taint module.ec2.aws_instance.ops` then `terraform apply`.

SSM Run Command exits with code 127 Exit code 127 means "executable not found". This is almost always caused by a Windows carriage return (`\r`) in the shebang line. Check the SSM document with `aws ssm get-document --name cloud-ops-lab-dev-health-check` and look for `\r` at the end of lines. The Terraform SSM module normalises line endings before creating the document — re-running `terraform apply` recreates the documents with corrected content.

GitHub Actions: "Given variables file dev.tfvars does not exist" `environments/dev/dev.tfvars` must be committed to the repository. Run `git ls-files environments/dev/dev.tfvars` — if it returns nothing, the file is either missing or excluded by `.gitignore`. Verify `.gitignore` does not contain a broad `*.tfvars` rule; it should only list `terraform.tfvars` and `*.auto.tfvars`.

Apply workflow OIDC authentication fails The apply IAM role trust policy uses `StringEquals` on the OIDC `sub` claim. The job must declare `environment: dev` (exact case, no spaces). Confirm the GitHub Environment is named `dev` in repository Settings and that `AWS_APPLY_ROLE_ARN` is set as a repository secret.